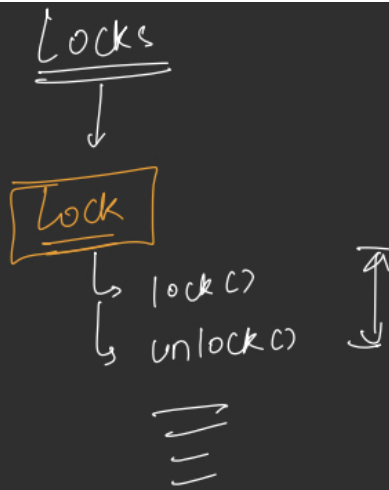




Limitations of Synchronized Block

① No control over lock.

Synchronized void f1() {

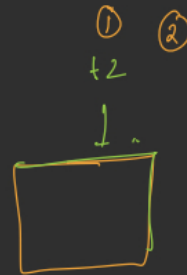
≡

}

② No timeout.

2 sec

③ No Fairness. (starvation).



Lock

↳ lock()

↳ unlock()

↳ Reentrant Lock

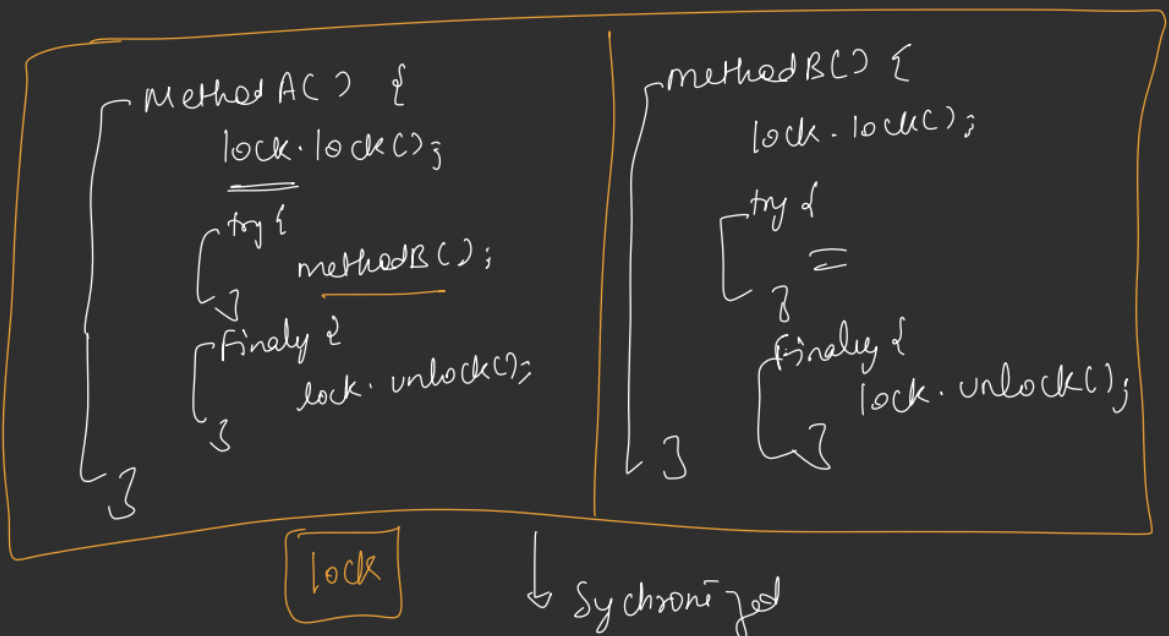
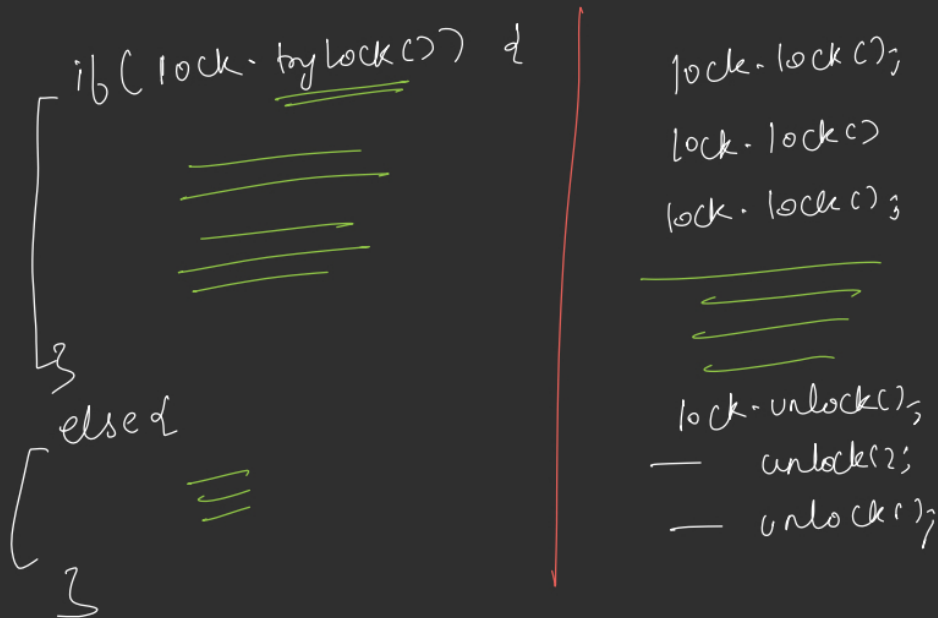
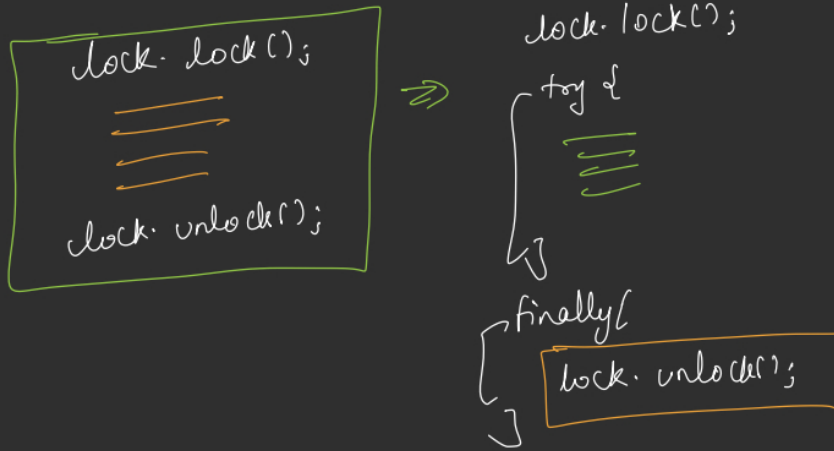
↳ Readwrite lock

↳ StampedLock

↳ Semaphore

Reentrantlock → (same thread can acquire lock multiple times)

Lock lock = new Reentrantlock();



Reentrant lock

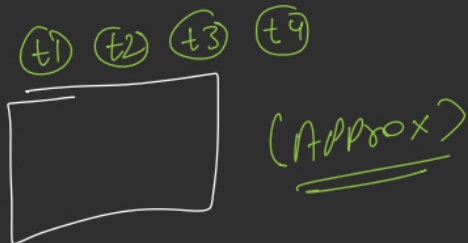
- ↳ lock()
- ↳ unlock()
- ↳ trylock()
- ↳ trylock(Timeout, TimeUnit)

↳ trylock(2, TimeUnit.SECONDS) {

}
else {
}

- isLocked()
- isHeldByCurrentThread()
- getHoldCount()
- isFair()

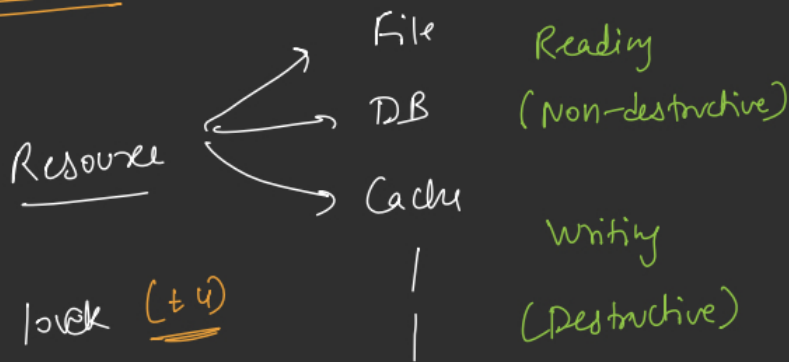
Reentrantlock
↳ Parent thread
→ Count ≤



Reentrantlock l1 = new Reentrantlock();

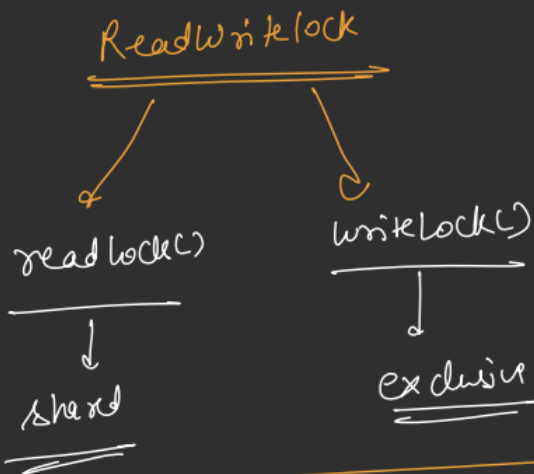
l2 = new Reentrantlock(true);

ReadWrite lock



Exclusive lock (± 4)

Shared lock (t_1, t_2, t_3)

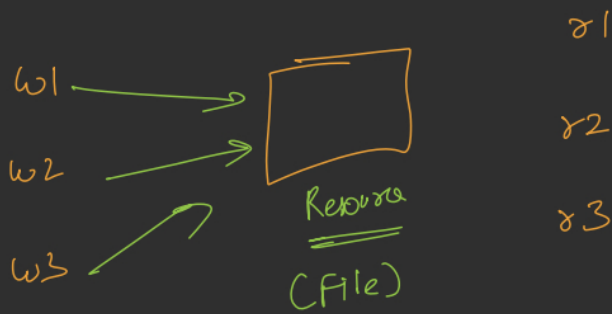


ReadWrite lock lock = new ReentrantReadWriteLock();

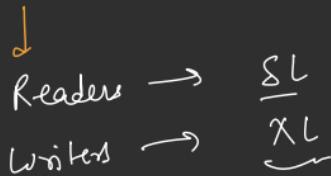
Multiple readers $\rightarrow$ Allowed together
One writer $\rightarrow$ Single Allowed
1w + 1R $\rightarrow$ X

lock r2 = lock.readLock();
lock w2 = lock.writeLock();

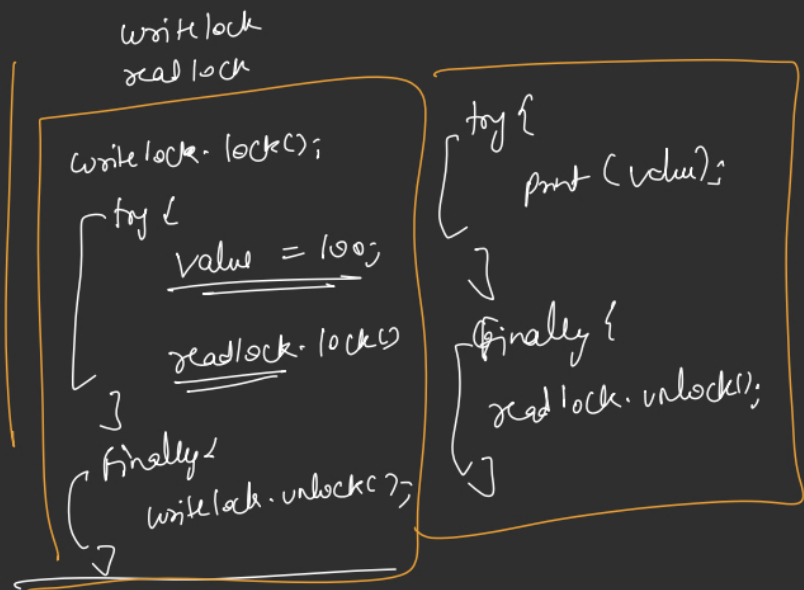
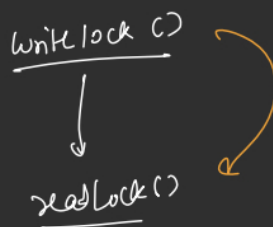
Reader - Writer Problem



Starvation



lock downgrading



StampedLock

↳ Modern version of ReadWriteLock.

- `writeLock()`
- `readLock()`
- `tryOptimisticRead()`

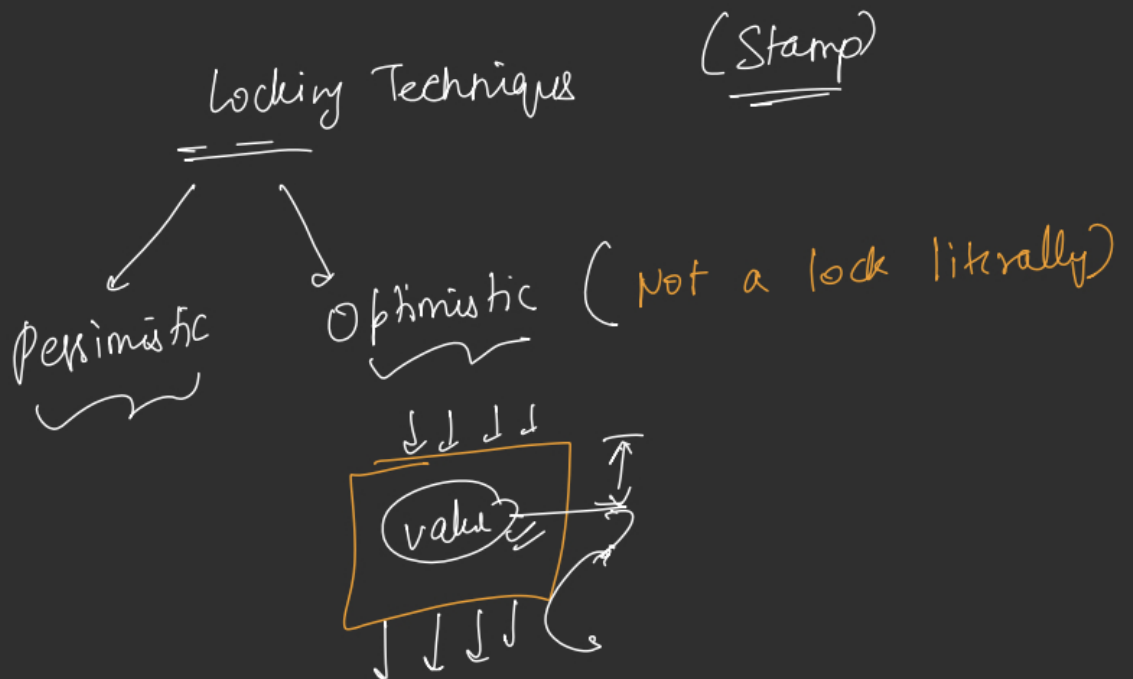
```
StampedLock lock = new StampedLock();
```

```
long stamp = lock.readLock(); // writeLock();
```

```
try {  
    =  
}
```

```
finally {  
    lock.unlockRead(stamp);  
    unlockWrite(stamp);  
}
```

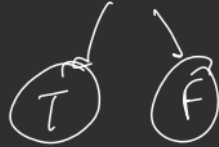
Optimistic Read



try Optimistic Read ()

- ① Get a stamp.
- ② Read data WITHOUT locking.
- ③ Checking if data is Modified.

⇒ if (lock.validate (stamp))



Stamped Lock



DEADLOCK.

Semaphore → permits = ③



Semaphore s1 = new Semaphore (3);

- s1.acquire() (Take 1 permit).
- s1.release() (Release 1 permit).

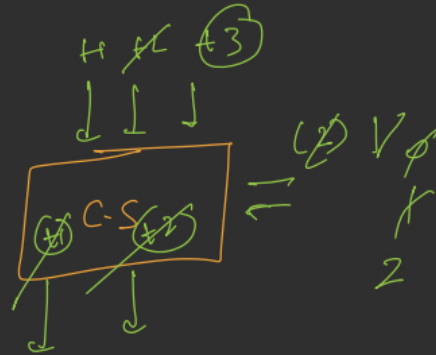
Semaphore

Binary Semaphore s1 = new (1);
→ Counting

Difference (lock v/s Semaphore).

lock : Ownership

semaphore : No Ownership.



Semaphore s1 = new Semaphore(3, true);

→ API Rate limiter.

→ parallel task limit.

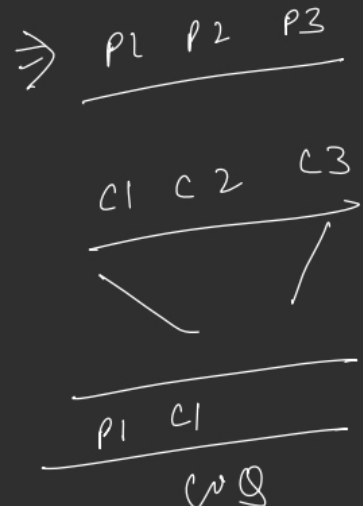
Condition → A waiting Area.



lock l1 = new ReentrantLock();

Condition c1 = l1.newCondition();

$\left. \begin{array}{l} c1.await() \\ c1.signal() \\ c1.signalAll() \end{array} \right\}$



Spurious Wakeup //



```
while (value != null) {  
    Z  
}
```